Rockchip 语音识别（RockASR）SDK开发指南

文件标识：RK-YH-YF-426

发布版本：V1.0.0

日期：2025-06-20

文件密级：□绝密　□秘密　□内部资料　■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：　福建省福州市铜盘路软件园A区18号

网址：　www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

RockASR SDK是基于Rockchip芯片平台实现“语音输入文本输出”的语音识别算法，当前提供以下算法功能

- 支持非流式整段识别：对完整音频文件进行一次性识别处理
- 支持实时流式识别：对持续输入的语音流进行实时识别，流式返回识别结果
- 支持自定义热词功能：允许添加特定词汇作为“热词”，以强化特定词汇识别效果

产品版本

芯片名称	系统版本
RK3576	Android/Linux
RK3588	Android/Linux

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	AI	2025.06.20	初始文档

目录

Rockchip 语音识别（RockASR）SDK开发指南

- 算法SDK功能
- 算法性能
- 算法SDK包说明
- 算法SDK包说明
- 授权说明
  - 设备上授权
  - PC上授权
- 算法SDK开发
  - Demo运行
  - 算法SDK调用
    - 非流式
    - 流式
- API参考
  - RockX2接口说明
  - RockASR SDK接口
    - 算法模块
      - ASR

结构体定义
RockXAsrInitParam
RockAsrResult
枚举定义
RockAsrState
RockAsrResultType
RockAsrLanguage
RockAsrWorkMode

算法SDK功能

RockASR SDK支持中英文语音内容的识别，能够将输入的语音数据转化为对应的文字信息，并提供灵活的识别模式（流式和非流式）和热词增强识别功能。

算法性能

RockASR SDK流式处理耗时和系统资源占用。

芯片平台	首包耗时（s）	实时率（RTF）	内存占用（MB）
RK3576	3.25	0.38	912.6
RK3588	3.18	0.46	819.2

算法SDK包说明

算法SDK包说明

使用SDK包应先获取到以下：
RK35XX_RockASR_SDK_V1.0.0_20250621.zip （rockasr）
RK35XX_ROCKX2_SDK_V1.0.0_2025XXXX.zip （ROCKX2：链接: https://console.box.lenovo.com/l/k1qDXS 提取码: rockx)

将ROCKX2 SDK压缩包解压至rockasr目录下，例如

```
├── build-android.sh
├── build-linux.sh
├── CMakeLists.txt
├── data
├── demo
├── doc
├── lib
├── README.md
└── RK3576_ROCKX2_SDK_V1.0.0_20250621
```

在CMakeLists.txt 配置ROCKX2路径

```
# rockx2
set(RockX2_DIR
${CMAKE_SOURCE_DIR}/RK3576_ROCKX2_SDK_V1.0.0_20250621/lib/${CMAKE_SYSTEM_NAME})
find_package(RockX2 REQUIRED)
```

授权说明

设备上授权

授权工具在ROCKX2_SDK目录下

- 设备联网
- 授权工具推到设备

```
adb push rkauth_tool/Android/arm64-v8a/rkauth_tool_bin /data/
```

- 执行授权程序

```
adb shell
cd /data/
./rkauth_tool_bin -u <user> -p <password> -o /data/key.lic
```

PC上授权

- 获取设备信息

```
adb push rkauth_tool/Android/arm64-v8a/rkdevice_info /data/
adb shell
/data/rkdevice_info /data/device.inf
```

将`device.inf`文件拉回PC上

- PC上执行授权程序

```
rkauth_tool_bin -u <user> -p <password> -o ./key.lic -d device.inf -m asr
```

该命令会生成授权文件，确保程序运行时能找到授权配置文件（默认为 ./demo/rockx_auth_config.json）

授权工具详细使用方法可以参考授权工具使用文档《Rockchip_User_Guide_RKAUTH_CN》

算法SDK开发

Demo运行

demo位于`demo/rockasr_demo`目录下

linux RK3576

编译

```
./build-linux.sh
```

部署到设备

```
adb push install/rockasr_linux_aarch64_rk3576 /data/
```

参考授权说明获取license文件，放到`./demo/rockx_auth_config.json`指定的路径

运行

示例demo用法：

```
Usage: ./rockasr_demo <wav_file> <language>
```

```
adb shell
cd /data/rockasr_linux_aarch64_rk3576/demo/rockasr_demo
export LD_LIBRARY_PATH=../../lib
./rockasr_demo ./test_zh_t10.wav 1
```

如需启用热词，需要额外传入一个热词列表的文本路径。热词文件为纯文本格式，每行一个需要增强识别效果的关键词，如demo/rockasr_demo/hot_word_list.txt

```
Usage: ./rockasr_demo <wav_file> <language> <hot_word_txt_path>
```

```
adb shell
cd /data/rockasr_linux_aarch64_rk3576/demo/rockasr_demo
export LD_LIBRARY_PATH=../../lib
./rockasr_demo ./test_zh_t10.wav 1 hot_word_list.txt
```

算法SDK调用

当前SDK支持RockX2框架C API接口进行调用，调用参考代码如下：

非流式

同步执行，阻塞至执行完成才会返回结果

```c
    // 创建实例
    RockXHandle handle = RockXCreate(0, NULL);

    // 配置参数
    RockXAsrInitParam* init_param = (RockXAsrInitParam*)malloc(sizeof(RockXAsrInitParam));
    if (init_param == NULL) {
        printf("init_param is NULL\n");
        return -1;
    }
    memset(init_param, 0, sizeof(RockXAsrInitParam));
    init_param->sampleRate = sample_rate;
```

```
    init_param->modelName = strdup("ASR_SS_ZH_EN");

    if (language == 1) {
        init_param->sourceLanguage = ROCKASR_LANG_CHINESE;
        init_param->targetLanguage = ROCKASR_LANG_ENGLISH;
    } else {
        init_param->sourceLanguage = ROCKASR_LANG_ENGLISH;
        init_param->targetLanguage = ROCKASR_LANG_CHINESE;
    }
    init_param->resultCallback = AsrResultCallback;
    init_param->userdata = NULL;
    init_param->workMode = ROCKASR_MODE_TRANSCRIBE;

    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "ASR", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_DATA_PATH, "/data/rockx2_rockasr_demo", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_AUTH_CONFIG,
"/data/rockx2_rockasr_demo/rockx_auth_config.json", 0);
    RockXSetParamInt(handle, NULL, "src_sample_rate",  sample_rate);
    // 控制音频分块的最大时长
    RockXSetParamInt(handle, NULL, "max_chunk_duration", -1); // 可设置 10s，20s，30s，如果设
置-1,则内部自动分配
    // 配置热词文本路径
    if (hot_word_path != NULL) {
        printf("hot_word_path %s\n",hot_word_path );
        RockXSetParamString(handle, NULL, "hot_word_txt_path", hot_word_path, 0);
    }
    RockXParamCallback cb;
    cb.OnRelease = ParamReleaseCallback;
    RockXSetParamPointer(handle, "ASR", "param", init_param, 1, &cb);

    // 初始化
    int ret = RockXInit(handle);
    if (ret != ROCKX_RET_SUCCESS) {
        printf("RockXInit fail, ret=%d\n", ret);
        return ret;
    }

    const char* out_names[1] = {
        "asr_result"
    };

    // 准备输入的音频数据
    RockXInput* input = RockXInputCreate(handle);
    RockXOutput* output = RockXOutputCreate(handle, out_names, 1);

    int16_t* buffer = (int16_t*)malloc(sample_num * channels * sizeof(int16_t));
    int read_size = WavReaderRead(wav_reader, buffer, sample_num * channels *
sizeof(int16_t));
    int read_sample_num = read_size / channels / sizeof(int16_t);

    RockXAudioWav wav_input;
    memset(&wav_input, 0, sizeof(RockXAudioWav));
    wav_input.sampleNum = read_sample_num;
```

```
    wav_input.data = buffer;
    wav_input.dtype = ROCKX_DATA_TYPE_INT16;
    wav_input.channels = channels;
    wav_input.sampleRate = sample_rate;
    wav_input.state = ROCKX_STREAM_STATE_FIRST;
    RockXPacket* audio_wav_packet = RockXInputGetPacketAt(input, 0);
    RockXPacketSetAudio(audio_wav_packet, &wav_input);
    // 注册回调释放
    RockXPacketSetReleaseCallback(audio_wav_packet, AudioReleaseCallback);

    // 执行
    RockXProcess(handle, input, output);

    RockXInputDestroy(handle, input);
    RockXOutputDestroy(handle, output);
```

注意：非流式输入wav_input.state 应该为 ROCKX_STREAM_STATE_FIRST

流式

异步执行，提交输入后立即返回，执行结果通过回调函数返回

```
    // 创建实例
    RockXHandle handle = RockXCreate(1, NULL);

    // 配置参数
    RockXAsrInitParam* init_param = (RockXAsrInitParam*)malloc(sizeof(RockXAsrInitParam));
    if (init_param == NULL) {
        printf("init_param is NULL\n");
        return -1;
    }
    memset(init_param, 0, sizeof(RockXAsrInitParam));
    init_param->sampleRate = sample_rate;
    init_param->modelName = strdup("ASR_SS_ZH_EN");

    if (language == 1) {
        init_param->sourceLanguage = ROCKASR_LANG_CHINESE;
        init_param->targetLanguage = ROCKASR_LANG_ENGLISH;
    } else {
        init_param->sourceLanguage = ROCKASR_LANG_ENGLISH;
        init_param->targetLanguage = ROCKASR_LANG_CHINESE;
    }
    init_param->resultCallback = AsrResultCallback;
    init_param->userdata = NULL;
    init_param->workMode = ROCKASR_MODE_STREAM;

    RockXSetParamString(handle, NULL, ROCKX_KEY_MODULE_NAME, "ASR", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_DATA_PATH, "/data/rockx2_rockasr_demo", 0);
    RockXSetParamString(handle, NULL, ROCKX_KEY_AUTH_CONFIG,
"/data/rockx2_rockasr_demo/rockx_auth_config.json", 0);
    RockXSetParamInt(handle, NULL, "src_sample_rate",  sample_rate);
    RockXParamCallback cb;
```

```
    cb.OnRelease = ParamReleaseCallback;
    RockXSetParamPointer(handle, "ASR", "param", init_param, 1, &cb);
    if (hot_word_path != NULL) {
        RockXSetParamString(handle, NULL, "hot_word_txt_path", hot_word_path, 0);
    }

    // 初始化
    int ret = RockXInit(handle);

    if (ret != ROCKX_RET_SUCCESS) {
        printf("RockXInit fail, ret=%d\n", ret);
        return ret;
    }

    const char* out_names[1] = {
        "asr_result"
    };
    RockXSetOutputInfo(handle, "out", out_names, 1, tmpRockXResultCallback,
ROCKX_CALLBACK_ON_SCHED_THREAD);

    // 异步
    RockXAudioWav wav_input;
    unsigned long cur_ts = 0;

    // 模拟流式输入
    int chunk_size = 20 * sample_rate / 1000;  // 20ms

    for (int i = 0; i < sample_num; i += chunk_size) {
        int16_t* buffer = (int16_t*)malloc(chunk_size * channels * sizeof(int16_t));
        int read_size = WavReaderRead(wav_reader, buffer, chunk_size * channels *
sizeof(int16_t));
        int read_sample_num = read_size / channels / sizeof(int16_t);

        RockXInput* input = RockXInputCreate(handle);

        RockXPacket* input_packet = RockXInputGetPacketAt(input, 0);
        RockXStreamState state = ROCKX_STREAM_STATE_CONTINUE;
        if (i == 0) {
            state = ROCKX_STREAM_STATE_FIRST;
        } else if (i + chunk_size >= sample_num) {
            state = ROCKX_STREAM_STATE_LAST;
        }
        RockXAudioWav wav_input;
        memset(&wav_input, 0, sizeof(RockXAudioWav));
        wav_input.sampleNum = read_sample_num;
        wav_input.data = buffer;
        wav_input.dtype = ROCKX_DATA_TYPE_INT16;
        wav_input.channels = channels;
        wav_input.sampleRate = sample_rate;
        wav_input.state = state;
        RockXPacketSetAudio(input_packet, &wav_input);
        // 注册回调释放
        RockXPacketSetReleaseCallback(input_packet, AudioReleaseCallback);
```

```
        RockXProcessAsync(handle, input, NULL);
        // 模拟流式输入
        usleep(20*1000);
    }
```

注意：流式处理下，wav_input.state需要按照时间情况设置，例如 首帧为ROCKX_STREAM_STATE_FIRST，中间帧为ROCKX_STREAM_STATE_CONTINUE，最后帧 ROCKX_STREAM_STATE_LAST

API参考

RockX2接口说明

RockX2算法框架接口请参考文档《Rockchip_Developer_Guide_ROCKX2_SDK_CN》

RockASR SDK接口

算法模块

模块名称	说明
ASR	语音识别算法模型

ASR

【说明】

语音识别算法

【输入】

输入名称	类型	说明
audio	RockXAudioWav	输入语音帧（参考RockX2文档）

【输出】

输出名称	类型	说明
asr_result	RockAsrResult	语音识别结果

【参数】

参数名称	类型	说明
ROCKX_KEY_MODULE_NAME	const char*	算法模块名称
ROCKX_KEY_DATA_PATH	const char*	模型目录路径
ROCKX_KEY_AUTH_CONFIG	const char*	License文件路径

参数名称	类型	说明
src_sample_rate	int	输入音频的采样率（仅支持16k）
max_chunk_duration	int	音频分块的最大时长（仅用于非流式）
hot_word_txt_path	const char*	热词文本路径
param	RockXAsrInitParam	Asr初始化参数

结构体定义

结构体名称	说明
RockXAudioWav	音频
RockXAsrInitParam	初始化参数
RockAsrResult	识别结果
RockAsrState	Asr结果状态
RockAsrResultType	识别结果类型
RockAsrLanguage	语言
RockAsrWorkMode	工作模式

RockXAsrInitParam

【说明】

Asr初始化参数

【声明】

```
typedef struct RockXAsrInitParam {
    char* modelName;
    RockAsrWorkMode workMode;
    RockAsrLanguage sourceLanguage;
    RockAsrLanguage targetLanguage;
    int sampleRate;
    void* userdata;
    RockAsrResultCallback resultCallback;
} RockXAsrInitParam;
```

【字段】

字段	类型	说明
modelName	char*	模型名称（仅支持ASR_SS_ZH_EN）

字段	类型	说明
workMode	RockAsrWorkMode	工作模式
sourceLanguage	RockAsrLanguage	源语言
targetLanguage	RockAsrLanguage	目标语言
sampleRate	int	输入音频采样率
userdata	void*	用户自定义数据
resultCallback	RockAsrResultCallback	结果回调函数

RockAsrResult

【说明】

识别结果

【声明】

```
typedef struct RockAsrResult {
    RockAsrResultType type;
    RockAsrState state;
    unsigned long tsStart;
    unsigned long tsEnd;
    char* result;
    char* newResult;
    char* transResult;
} RockAsrResult;
```

【字段】

字段	类型	说明
type	RockAsrResultType	结果类型
state	RockAsrState	结果状态
tsStart	unsigned lon	开始时间戳
tsEnd	unsigned long	结束时间戳
result	char*	当前句子识别结果（非流式返回的结果）
newResult	char*	新的识别结果（流式模式返回的结果）
transResult	char*	翻译结果

枚举定义

枚举	描述
RockAsrState	Asr结果状态
RockAsrResultType	识别结果类型
RockAsrLanguage	语言
RockAsrWorkMode	工作模式

RockAsrState

【描述】

Asr结果状态

【枚举值】

枚举	说明
ASR_STATE_NONE	无（0）
ASR_STATE_FIRST	开始（1）
ASR_STATE_RUNNING	运行中（2）
ASR_STATE_FINISH	结束（3）

RockAsrResultType

【描述】

识别结果类型

【枚举值】

枚举	说明
ROCKASR_RESULT_TYPE_TRANSCRIPT	ASR识别的结果（0）
ROCKASR_RESULT_TYPE_TRANSLATE	翻译（1）
ROCKASR_RESULT_TYPE_ALL	同时包含ASR和翻译结果（2）

RockAsrLanguage

【描述】

语言

【枚举值】

枚举	说明
ROCKASR_LANG_ENGLISH	英文（0）
ROCKASR_LANG_CHINESE	中文（1）

RockAsrWorkMode

【描述】

工作模式

【枚举值】

枚举	说明
ROCKASR_MODE_STREAM	流式（0）
ROCKASR_MODE_TRANSCRIBE	非流式（1）
